

Dart Programming

Creating a Hello World App

Lesson 2:



Creating a New App

- Start Android Studio.
- If you have a project open, click the menu bar and select File ⇨ New ⇨ New Flutter Project. If no project is open, click Start A New Flutter Project.
- Select Flutter Application.
- The customary naming convention is to separate words with an underscore.
- Enter the required details in the screen.
- On the same screen for the Platform Channel Language, select both the options for Java and Objective-C.
- Click the Finish button.

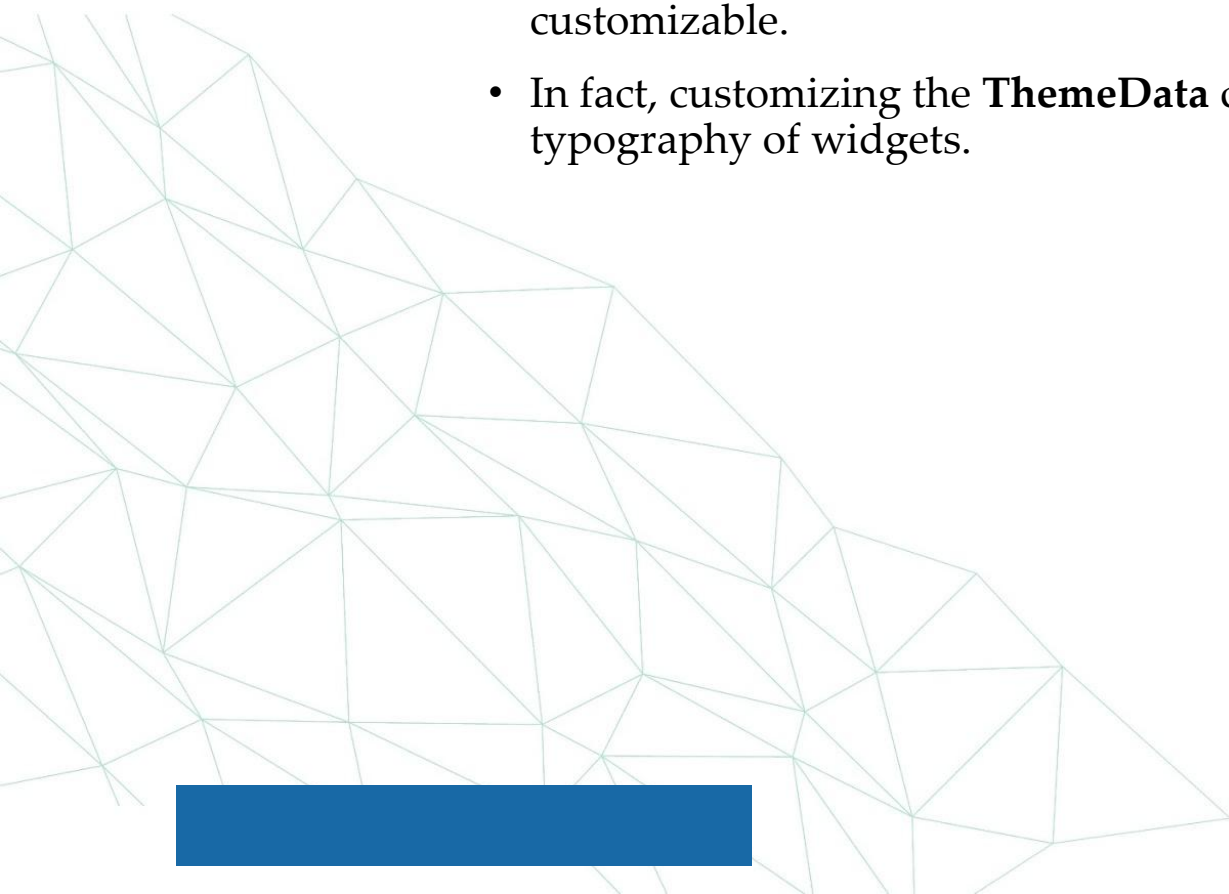
USING HOT RELOAD

- Flutter's hot reload helps you see code and user interface changes immediately while retaining state to an app running the Dart virtual machine.
- In other words, every time you make code changes, you don't need to reload the app, because the current page shows the changes immediately.
- This is an incredible time-saving feature for any developer.



USING THEMES TO STYLE YOUR APP

- The theme widgets are a great way to style and define global colors and font styles for your app.
- There are two ways to use theme widgets—to style the look and feel globally or to style just a portion of the app.
- The beauty of Flutter is that most items are widgets, and just about everything is customizable.
- In fact, customizing the **ThemeData** class allows you to change the color and typography of widgets.



Using a Global App Theme

- The current color is blue, so let's change it to light green. Add a new line below the primarySwatch and add code to change the background color (canvasColor) to lightGreen.
 - primarySwatch: Colors.blue,
 - // Change it to
 - primarySwatch: Colors.lightGreen,
 - canvasColor: Colors.lightGreen.shade100,
- To show a little Flutter awesomeness, add a platform property of TargetPlatform.iOS after the canvasColor property, and run the app from the Android emulator.
- Suddenly, the iOS traits are running on Android. The app bar's title is not left aligned but changed to the center, which is the customary iOS style.
 - primarySwatch: Colors.lightGreen,
 - canvasColor: Colors.lightGreen.shade100,
 - platform: TargetPlatform.iOS

Using a Global App Theme

- Google intended to let developers know that the app performance is not in release mode.
- Flutter builds a debug version of the app, and performance is slower.
- Using release (device only) mode, it creates an app with speed optimization.
- Add the `debugShowCheckedModeBanner` property and set the value to false, and the red debug banner is removed.
- Turning off the red debug banner is for aesthetic purposes only; you are still running a debug version of the app.
 - `return new MaterialApp(`
 - `debugShowCheckedModeBanner: false,`
 - `title: 'My Counter',`
 - `theme: new ThemeData(`
 - `...`

Using a Theme for Part of an App

- To override the app-wide theme, you can wrap widgets in a Theme widget. This method will completely override the app **ThemeData** instance without inheriting any styles.
- We override the default theme with a Theme widget that uses the data property to customize **ThemeData**, and the child widget uses the data property to customize the colors.
- Use the **copyWith** method to create a copy of the app parent theme and replace only the properties that you need to change.
- Breaking it down, **Theme.of(context).copyWith()** extends the parent Theme, and you can override the properties needed **inside copyWith(cardColor: Colors.deepOrange)**.
 - body: Center(
 - child: Theme(
 - data: Theme.of(context).copyWith(cardColor: Colors.deepOrange),
 - child: Card(
 - child: Text('copyWith Theme'),
 -),),),

UNDERSTANDING STATELESS AND STATEFUL WIDGETS

- StatelessWidget is used when data does not change, and it relies on the initial information.
- It's a widget without state, and the values are final. Some examples are Text, Button, Icon, and Image.
 - class Instructions extends StatelessWidget {
 - @override
 - Widget build(BuildContext context) {
 - return Text('When using a StatelessWidget...');
 - }
 - }

UNDERSTANDING STATELESS AND STATEFUL WIDGETS

- StatefulWidget is used when data changes. It's a widget with state that might change over time, requiring two classes.
- For changes to propagate to the UI, making a call to the setState() method is necessary.
 - StatefulWidget class—This creates an instance of the State class.
 - State class—This is for data that can be read synchronously when the widget is built and might change over time.
 - setState()—From within the State class, you make a call to the setState() method to refresh changed data, telling the framework that the widget should redraw because the state has changed.
 - For all the variables that need changes, modify the values in setState(() {
 - _myValue += 50.0;}).
 - Any variable values modified outside the setState() method will not refresh the UI.
 - Therefore, it is best to place calculations that do not need state changes outside the setState() method.

UNDERSTANDING STATELESS AND STATEFUL WIDGETS

- StatefulWidget is used when data changes. It's a widget with state that might change over time, requiring two classes.
- For changes to propagate to the UI, making a call to the setState() method is necessary.
 - StatefulWidget class—This creates an instance of the State class.
 - State class—This is for data that can be read synchronously when the widget is built and might change over time.
 - setState()—From within the State class, you make a call to the setState() method to refresh changed data, telling the framework that the widget should redraw because the state has changed.
 - For all the variables that need changes, modify the values in setState(() {
 - _myValue += 50.0;}).
 - Any variable values modified outside the setState() method will not refresh the UI.
 - Therefore, it is best to place calculations that do not need state changes outside the setState() method.

UNDERSTANDING STATELESS AND STATEFUL WIDGETS

- Consider the example of a page that shows your maximum bid on a product. Every time the Increase Bid button is tapped, your bid increases by \$50.
 - You begin by creating a `MaximumBid` class that extends the `StatefulWidget` class.
 - Create a `_MaximumBidState` class that extends the state of the `MaximumBid` class.
 - In the `_MaximumBidState` class, you declare a variable named `_maxBid`.
 - The `_increaseMyMaxBid()` method calls the `setState()` method, which increments the `_maxBid` value by \$50.
 - The UI consists of a `Text` widget that shows the 'My Maximum Bid: \$_maxBid' value and a `FlatButton` with an `onPressed` property that calls the `_increaseMyMaxBid()` method.
 - The `_increaseMyMaxBid()` method executes the `setState()` method, which adds \$50 to the `_maxBid` variable, and the `Text` widget amount is redrawn.

UNDERSTANDING STATELESS AND STATEFUL WIDGETS

- `class MaximumBid extends StatefulWidget {`
- `@override`
- `_MaximumBidState createState() => _MaximumBidState();`
- `class _MaximumBidState extends State<MaximumBid> {`
- `double _maxBid = 0.0;`
- `void _increaseMyMaxBid() {`
- `setState(() {`
- `_maxBid += 50.0;`
- `});`
- `}`
- `@override`
- `Widget build(BuildContext context) {`
- `return Column(`
- `children: <Widget>[`
- `Text('My Maximum Bid: $_maxBid'),`
- `FlatButton.icon(`
- `onPressed: () => _increaseMyMaxBid(),`
- `icon: Icon(Icons.add_circle),`
- `label: Text('Increase Bid'),),],);`

USING EXTERNAL PACKAGES

- Sometimes it's not worth building a widget from scratch. Flutter supports third-party packages for the Flutter and Dart ecosystems.
- Packages contain the source code logic and are easily shared.
- There are two types of packages, Dart and plugin.
 - **Dart packages** are written in Dart and may contain Flutter-specific dependencies.
 - **Plugin packages** are written in Dart (with the Dart code exposing the API) but are combined with platform-specific code implementations for Android (Java or Kotlin) and/or iOS (Objective-C or Swift).
 - Most plugin packages aim to support both Android and iOS.

Searching for Packages

- In the app, say you need to store the user preferences on both iOS and Android and want to find a package to do that for you.
 - Start your web browser and navigate to <https://pub.dartlang.org/flutter>. Packages are published at this location often by other developers and Google.
 - Click the Search Flutter Packages search bar. Enter shared preferences, and the results will be sorted by relevance.
 - Click the link for the shared_preferences package. (The direct link is https://pub.dartlang.org/packages/shared_preferences.)
 - Details on how to install and use the shared_preferences package are available at this location. The Flutter team authors this particular package.
 - Click the Installing tab for detailed instructions. Each package has instructions on how to install and use it.
 - Most of the packages' installation is similar, but they differ on how to use and implement the code.
 - The instructions are located on the home page of each package.

Searching for Packages

- to implement the `shared_preferences` external package in your app.
 - Open the project app with Android Studio.
 - Open the `pubspec.yaml` file by double-clicking.
 - In the dependencies: section, add `shared_preferences: ^0.5.1+1`. (Your version might be higher.)
 - Save the file, and the package will install. If you do not see the process automatically run, you can manually invoke it by entering `flutter packages get` in your Terminal window in Android Studio.
 - `Flutter pub add shared_preferences`
 - Once finished, the message will say Process finished with exit code 0.
 - Import the package in the `main.dart` file after the `material.dart` import line, located at the top of the file. Save the changes.
 - `import 'package:flutter/material.dart';`
 - `import 'package:shared_preferences/shared_preferences.dart';`

Using Packages

- Each package has its unique way of being implemented. It's always good to read the documentation.
- For the `shared_preferences` package, you need to add a few lines to implement it.
 - In the `_MyHomePageState` class, add a function called `_updateSharedPreferences()`.
 - `class _MyHomePageState extends State<MyHomePage> {`
 - `// ...`
 - `void _updateSharedPreferences() async {`
 - `SharedPreferences prefs = await SharedPreferences.getInstance();`
 - `int counter = (prefs.getInt('counter') ?? 0) + 1;`
 - `print('Pressed $counter times.');`
 - `await prefs.setInt('counter', counter);`
 - `}`
 - `// ...`
 - `}`

Lesson 2:

Creating a Starter Project Template

CREATING AND ORGANIZING FOLDERS AND FILES

- Create a new Flutter project.
- It's time to create the folder structure to keep you organized. Click the Terminal button at the bottom of the Android Studio window.
- To create the folder structures, execute **the `mkdir -p folder/subfolder` command**.
- Run each **mkdir** command in the Terminal window to create each folder structure. For example:
 - `\starter_exercise>mkdir assets\images`
 - `\starter_exercise>mkdir lib\pages`
 - `\starter_exercise>mkdir lib\models`
 - `\starter_exercise>mkdir lib\utils`
 - `\starter_exercise>mkdir lib\widgets`
 - `\starter_exercise>mkdir lib\services`

CREATING AND ORGANIZING FOLDERS AND FILES

- **assets/images:** The assets folder holds subfolders such as images, fonts, and configuration files.
- **lib/pages:** The pages folder holds user interface (UI) files such as logins, lists of items, charts, and settings.
- **lib/models:** The models folder holds classes for your data such as customer information and inventory items.
- **lib/utills:** The utills folder holds helper classes such as date calculations and data conversion.
- **lib/widgets:** The widgets folder holds different Dart files separating widgets to reuse through the app.
- **lib/services:** The services folder holds classes that help to retrieve data from services over the Internet. A great example is when using Google Cloud Firestore, Cloud Storage, Realtime Database, Authentication, or Cloud Functions. You can retrieve data from social media accounts, database servers, and so on.

STRUCTURING WIDGETS

- Before you start developing an app, it's important to create your structure; like when building a house, the foundation is created first.
- When creating a new Flutter project, the software development kit (SDK) does not automatically create the separate home.dart file, which contains the main presentation page when the app starts.
- Therefore, to have code separation, you must manually create the pages folder and the home.dart file inside it.
- The main.dart file contains the main() function that starts the app and calls the Home widget in the home.dart file.

STRUCTURING WIDGETS

- A great way to learn how Flutter works is to start from a blank slate.
 - Delete all the contents of the main.dart file. The main.dart file has three main sections.
 - The import package/file
 - The main() function
 - A class that extends a StatelessWidget widget and returns the app as a widget.
 - start by adding the code to the main.dart file and saving it.
 - Import the package/file. The default import is the material.dart library to allow the use of Material Design.
 - Then import the home.dart page located in the pages folder.
 - `import 'package:flutter/material.dart';`
 - `import 'package:ch4_starter_exercise/pages/home.dart';`

STRUCTURING WIDGETS

- A great way to learn how Flutter works is to start from a blank slate.
 - Delete all the contents of the main.dart file. The main.dart file has three main sections.
 - The import package/file
 - The main() function
 - A class that extends a StatelessWidget widget and returns the app as a widget.
 - start by adding the code to the main.dart file and saving it.
 - Import the package/file. The default import is the material.dart library to allow the use of Material Design.
 - Then import the home.dart page located in the pages folder.
 - `import 'package:flutter/material.dart';`
 - `import 'package:ch4_starter_exercise/pages/home.dart';`

STRUCTURING WIDGETS

- After the two import statements, leave a blank line and enter the main() function listed next. The main() function is the entry point to the app and calls the MyApp class.
 - `void main() => runApp(MyApp());`
- Type the MyApp class that extends StatelessWidget.
- The MyApp class returns a MaterialApp widget declaring title, theme, and home properties.
 - `class MyApp extends StatelessWidget {`
 - `@override`
 - `Widget build(BuildContext context) {`
 - `return MaterialApp(`
 - `debugShowCheckedModeBanner: false,`
 - `title: 'Starter Template',`
 - `theme: ThemeData(`
 - `primarySwatch: Colors.blue,),home: Home(),);`
 - `}}`

STRUCTURING WIDGETS

- Create a new Dart file in the pages folder. Right-click the pages folder, select New ⇒ Dart File, enter home.dart, and click the OK button to save.
- import the material.dart package/file.
 - `import 'package:flutter/material.dart';`
- Now give the StatefulWidget class its name: Home. Since it's a class, the naming convention is to start the word with an uppercase letter.
- `// home.dart`
 - `import 'package:flutter/material.dart';`
 - `class Home extends StatefulWidget {`
 - `@override`
 - `_HomeState createState() => _HomeState();`
 - `class _HomeState extends State<Home> {`
 - `@override`
 - `Widget build(BuildContext context) {`
 - `return Container();`
 - `}}`

Lesson 3:

Understanding the Widget Tree

- The widget tree is how you create your UI; you position widgets within each other to build simple and complex layouts.
- Since just about everything in the Flutter framework is a widget, and as you start nesting them, the code can become harder to follow.
- A good practice is to try to keep the widget tree as shallow as possible.
- To understand the full effects of a deep tree, you'll look at a full widget tree and then refactor it into a shallow widget tree, making the code more manageable.



Built-in Widgets

- Flutter has a big focus on the UI, and because of this, it contains a large catalog of built-in widgets that allow you to get started with a high-quality UI relatively easily.
- The available widgets of Flutter range from simple ones, such as the Text widget, to complex widgets that enable you to design dynamic UI with animations and multiple gesture handling.
- Some of the Built-in widgets are:
 - **Scaffold**—Implements the Material Design visual layout, allowing the use of Flutter's Material Components widgets
 - **AppBar**—Implements the toolbar at the top of the screen
 - **CircleAvatar**—Usually used to show a rounded user profile photo, but you can use it for any image
 - **Divider**—Draws a horizontal line with padding above and below

Built-in Widgets

- If the app you are creating is using Cupertino, you can use the following widgets instead. Note that with Cupertino you can use two different scaffolds, a page scaffold or a tab scaffold.
 - **CupertinoPageScaffold**—Implements the iOS visual layout for a page. It works with CupertinoNavigationBar to provide the use of Flutter's Cupertino iOS-style widgets.
 - **CupertinoTabScaffold**—Implements the iOS visual layout. This is used to navigate multiple pages, with the tabs at the bottom of the screen allowing you to use Flutter's Cupertino iOS-style widgets.
 - **CupertinoNavigationBar**—Implements the iOS visual layout toolbar at the top of the screen.

Built-in Widgets

- If the app you are creating is using Cupertino, you can use the following widgets instead. Note that with Cupertino you can use two different scaffolds, a page scaffold or a tab scaffold.
 - **CupertinoPageScaffold**—Implements the iOS visual layout for a page. It works with CupertinoNavigationBar to provide the use of Flutter's Cupertino iOS-style widgets.
 - **CupertinoTabScaffold**—Implements the iOS visual layout. This is used to navigate multiple pages, with the tabs at the bottom of the screen allowing you to use Flutter's Cupertino iOS-style widgets.
 - **CupertinoNavigationBar**—Implements the iOS visual layout toolbar at the top of the screen.

Built-in Widgets

- The following widgets can be used with both Material Design and Cupertino:
 - **SingleChildScrollView**—This adds vertical or horizontal scrolling ability to a single child widget.
 - **Padding**—This adds left, top, right, and bottom padding.
 - **Column**—This displays a vertical list of child widgets.
 - **Row**—This displays a horizontal list of child widgets.
 - **Container**—This widget can be used as an empty placeholder (invisible) or can specify height, width, color, transform (rotate, move, skew), and many more properties.
 - **Expanded**—This expands and fills the available space for the child widget that belongs to a Column or Row widget.
 - **Text**—The Text widget is a great way to display labels on the screen. It can be configured to be a single line or multiple lines. An optional style argument can be applied to change the color, font, size, and many other properties.

Built-in Widgets

- **Stack**—What a powerful widget! Stack lets you stack widgets on top of each other and use a Positioned (optional) widget to align each child of the Stack for the layout needed. A great example is a shopping cart icon with a small red circle on the upper right to show the number of items to purchase.
- **Positioned**—The Positioned widget works with the Stack widget to control child positioning and size. A Positioned widget allows you to set the height and width. You can also specify the position location distance from the top, bottom, left, and right sides of the Stack widget.

BUILDING THE FULL WIDGET TREE

- To create a page layout, you nest widgets to create a custom UI.
- The result of adding widgets together is called the widget tree.
- As the number of widgets increases, the widget tree starts to expand quickly and makes the code hard to read and manage.
- As the following example:
 - `import 'package:flutter/material.dart';`
 - `class Home extends StatefulWidget {`
 - `@override`
 - `_HomeState createState() => _HomeState();`
 - `}`
 - `class _HomeState extends State<Home> {`
 - `@override`
 - `Widget build(BuildContext context) {`
 - `return Scaffold(`
 - `appBar: AppBar(`
 - `title: Text('Widget Tree'),`
 - `),`

[illegible]

BUILDING THE FULL WIDGET TREE

```
• Padding(padding: EdgeInsets.all(16.0)),  
• Container(  
•   color: Colors.amber,  
•   height: 40.0,  
•   width: 40.0,  
• ),  
• Padding(padding: EdgeInsets.all(16.0)),  
• Container(  
•   color: Colors.brown,  
•   height: 20.0,  
•   width: 20.0,  
• ),  
• Divider(),  
• Row(  
•   children: <Widget>[  
•     CircleAvatar(  
•       backgroundColor: Colors.lightGreen,  
•       radius: 100.0,  
•     ),  
•   ],  
• ),
```

BUILDING THE FULL WIDGET TREE

```
• child: Stack(  
•     children: <Widget>[  
•         Container(  
•             height: 100.0,  
•             width: 100.0,  
•             color: Colors.yellow, ),  
•         Container(  
•             height: 60.0,  
•             width: 60.0,  
•             color: Colors.amber,),  
•         Container(  
•             height: 40.0,  
•             width: 40.0,  
•             color: Colors.brown,),],),),),  
•     Divider(),  
•     Text('End of the Line'),],),],),),),),);}}
```

BUILDING A SHALLOW WIDGET TREE

- To make the example code more readable and maintainable, we'll refactor major sections of the code into separate entities.
- we have multiple refactor options, and the most common techniques are constants, methods, and widget classes.



BUILDING A SHALLOW WIDGET TREE

- **Refactoring with a Constant**

- Refactoring with a constant initializes the widget to a final variable. This approach allows you to separate widgets into sections, making for better code readability.
- When widgets are initialized with a constant, they rely on the BuildContext object of the parent widget.
- Which means Every time the parent widget is redrawn, all the constants will also redraw their widgets, so you can't do any performance optimization.
- The following sample code shows how to use a constant to initialize the container variable as final with the Container widget. You insert the container variable in the widget tree where needed.
 - `final container = Container(`
 - `color: Colors.yellow,`
 - `height: 40.0,`
 - `width: 40.0,`
 - `);`

BUILDING A SHALLOW WIDGET TREE

- **Refactoring with a Method**

- Refactoring with a method returns the widget by calling the method name.
- The method can return a value by a general widget (Widget) or a specific widget (Container, Row, and others).
- The widgets initialized by a method rely on the BuildContext object of the parent widget.
- This approach allows you to separate widgets into sections, making for better code readability.
- Every time the parent widget is redrawn, all the methods will also redraw their widgets. That means the widget tree is not optimizable for performance.
- The following sample code shows how to use a method to return a Container widget. This first method returns the Container widget as a general Widget, and the second method returns the Container widget as a Container widget.

BUILDING A SHALLOW WIDGET TREE

- Both approaches are acceptable. You insert the
- `_buildContainer()` method name in the widget tree where needed.
- `// Return by general Widget Name`
- `Widget _buildContainer() {`
- `return Container(`
- `color: Colors.yellow,`
- `height: 40.0,`
- `width: 40.0,`
- `);}`
- `// Or Return by specific Widget like Container in this case`
- `Container _buildContainer() {`
- `return Container(`
- `color: Colors.yellow,`
- `height: 40.0,`
- `width: 40.0,`
- `);}`

BUILDING A SHALLOW WIDGET TREE

- To refactor the widgets, use the method pattern to flatten the widget tree.
 - Open the home.dart file.
 - Place the cursor on the first Row widget and right-click.
 - Select Refactor ⇨ Extract ⇨ Method and click Method.
 - In the Extract Method dialog, enter `_buildHorizontalRow` for the method name.
 - Notice the underscore before build; this lets Dart know that it is a private method.
 - Notice that the entire Row widget and the children are highlighted to make it easy to view the affected code.

BUILDING A SHALLOW WIDGET TREE

- Creating a shallow widget tree means each widget is separated into its own method by functionality.
- Keep in mind that how you separate widgets will be different depending on the functionality needed.
- Separating widgets by method improves code readability, but you lose the performance benefits of Flutter's subtree rebuilding.
- All the widgets in the method rely on the parent's BuildContext, meaning every time the parent is redrawn, the method is also redrawn.



BUILDING A SHALLOW WIDGET TREE

- **Refactoring with a Widget Class**
 - Refactoring with a widget class allows you to create the widget by subclassing the **StatelessWidget** class.
 - You can create reusable widgets within the current or separate Dart file and initiate them anywhere in the application.
 - Notice that the constructor starts with a **const** keyword, which allows you to cache and reuse the widget. When calling the constructor to initiate the widget, use the **const** keyword.
 - By calling with the **const** keyword, the widget does not rebuild when other widgets change their state in the tree.
 - If you omit the **const** keyword, the widget will be called every time the parent widget redraws.
 - The widget class relies on its own **BuildContext**, not the parent like the **constant** and **method** approaches. **BuildContext** is responsible for handling the location of a widget in the widget tree.

BUILDING A SHALLOW WIDGET TREE

- Every time the parent widget is redrawn, all the widget classes will not redraw.
- They are built only once, which is great for performance optimization.
- The following sample code shows how to use a widget class to return a Container widget. You insert the const ContainerLeft() widget in the widget tree where needed. Note the use of the const keyword to take advantage of caching.
 - class ContainerLeft extends StatelessWidget {
 - const ContainerLeft({
 - Key key,
 - }) : super(key: key);
 - @override
 - Widget build(BuildContext context) {
 - return Container(
 - color: Colors.yellow,
 - height: 40.0,
 - width: 40.0,
 -);}}

BUILDING A SHALLOW WIDGET TREE

- Refactoring with a Widget Class to Create a Shallow Widget Tree:
 - Open the home.dart file.
 - Place the cursor on the first Row widget and right-click.
 - Select Refactor ⇨ Extract ⇨ Flutter Widget.
 - In the Extract Widget dialog, enter RowWidget for the widget name.
 - The Row widget is replaced with the RowWidget() widget class.
 - Since the Row widget will not change state, add the const keyword before calling the RowWidget() class.
 - Scroll to the bottom of the code, and the widgets are nicely refactored into the RowWidget (StatelessWidget) class.
- Creating a shallow widget tree means each widget is separated into its own widget class by functionality.
- Keep in mind that how you separate widgets will be different depending on the functionality needed.